

Programação Competitiva

Aula 2.1: Complexidade de algoritmos

Alberto Tavares Duarte Neto

01 de Setembro, 2025

Universidade de Brasília

Definição

Sejam $f(n)$ e $g(n)$ funções não negativas. Definimos

$$O(g(n)) = \{f(n) \mid \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que} \\ 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\} .$$

Definição

Sejam $f(n)$ e $g(n)$ funções não negativas. Definimos

$$O(g(n)) = \{f(n) \mid \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que} \\ 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\} .$$

Se $f(n) \in O(g(n))$, dizemos que $g(n)$ limita superiormente $f(n)$.
Escrevemos $f(n) = O(g(n))$.

Definição

Sejam $f(n)$ e $g(n)$ funções não negativas. Definimos

$$O(g(n)) = \{f(n) \mid \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que} \\ 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}.$$

Se $f(n) \in O(g(n))$, dizemos que $g(n)$ limita superiormente $f(n)$.

Escrevemos $f(n) = O(g(n))$. Dizemos que $f(n) = \Theta(g(n))$ se $O(f(n)) = O(g(n))$.

Definição

Sejam $f(n)$ e $g(n)$ funções não negativas. Definimos

$$O(g(n)) = \{f(n) \mid \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que} \\ 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}.$$

Se $f(n) \in O(g(n))$, dizemos que $g(n)$ limita superiormente $f(n)$.

Escrevemos $f(n) = O(g(n))$. Dizemos que $f(n) = \Theta(g(n))$ se $O(f(n)) = O(g(n))$.

Exemplos

(i) $f(n) = 2n^2 + 3n + 7 = O(n^2)$.

Definição

Sejam $f(n)$ e $g(n)$ funções não negativas. Definimos

$$O(g(n)) = \{f(n) \mid \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que} \\ 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\} .$$

Se $f(n) \in O(g(n))$, dizemos que $g(n)$ limita superiormente $f(n)$.

Escrevemos $f(n) = O(g(n))$. Dizemos que $f(n) = \Theta(g(n))$ se $O(f(n)) = O(g(n))$.

Exemplos

(i) $f(n) = 2n^2 + 3n + 7 = O(n^2)$.

(ii) $\log_b(n) = O(\log_2(n))$ para qualquer $b > 1$, pois

$$\log_2(n) = \frac{\log_b(n)}{\log_b(2)} .$$

Proposição

Seja

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c .$$

Então

Proposição

Seja

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c .$$

Então

(i) se $c > 0$, então $f(n) = \Theta(g(n))$;

Proposição

Seja

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c .$$

Então

- (i) se $c > 0$, então $f(n) = \Theta(g(n))$;
- (ii) se $c = 0$, então $f(n) = O(g(n))$ e $g(n) \neq O(f(n))$;

Proposição

Seja

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c .$$

Então

- (i) se $c > 0$, então $f(n) = \Theta(g(n))$;
- (ii) se $c = 0$, então $f(n) = O(g(n))$ e $g(n) \neq O(f(n))$;
- (iii) se $c = \infty$, então $g(n) = O(f(n))$ e $f(n) \neq O(g(n))$;

Proposição

Seja

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c .$$

Então

- (i) se $c > 0$, então $f(n) = \Theta(g(n))$;
- (ii) se $c = 0$, então $f(n) = O(g(n))$ e $g(n) \neq O(f(n))$;
- (iii) se $c = \infty$, então $g(n) = O(f(n))$ e $f(n) \neq O(g(n))$;

Exemplo

Temos

$$\lim_{n \rightarrow \infty} \frac{4^n}{2^n} = \lim_{n \rightarrow \infty} 2^n = \infty ,$$

portanto $4^n \neq O(2^n)$. De forma geral, se $a > b$ são números reais positivos, então $a^n \neq O(b^n)$

Complexidade assintótica

Definição

A **complexidade computacional** de um algoritmo é a quantidade de recursos computacionais necessária para executá-lo.

Complexidade assintótica

Definição

A **complexidade computacional** de um algoritmo é a quantidade de recursos computacionais necessária para executá-lo.

Definição

A **complexidade computacional assintótica** de um algoritmo é a estimativa da complexidade computacional utilizando a análise assintótica.

Complexidade assintótica

Definição

A **complexidade computacional** de um algoritmo é a quantidade de recursos computacionais necessária para executá-lo.

Definição

A **complexidade computacional assintótica** de um algoritmo é a estimativa da complexidade computacional utilizando a análise assintótica.

Seja T a função tal que $T(n)$ é o número de operações de um determinado algoritmo dado uma entrada de tamanho n .

Complexidade assintótica

Definição

A **complexidade computacional** de um algoritmo é a quantidade de recursos computacionais necessária para executá-lo.

Definição

A **complexidade computacional assintótica** de um algoritmo é a estimativa da complexidade computacional utilizando a análise assintótica.

Seja T a função tal que $T(n)$ é o número de operações de um determinado algoritmo dado uma entrada de tamanho n . Analisamos assintoticamente T , ou seja, tentamos calcular $\Theta(T(n))$ ou encontrar funções f tais que $T(n) = O(f(n))$.

Complexidade assintótica

Definição

A **complexidade computacional** de um algoritmo é a quantidade de recursos computacionais necessária para executá-lo.

Definição

A **complexidade computacional assintótica** de um algoritmo é a estimativa da complexidade computacional utilizando a análise assintótica.

Seja T a função tal que $T(n)$ é o número de operações de um determinado algoritmo dado uma entrada de tamanho n . Analisamos assintoticamente T , ou seja, tentamos calcular $\Theta(T(n))$ ou encontrar funções f tais que $T(n) = O(f(n))$.

- (i) Iterar todos os elementos de um vetor de n elementos tem complexidade $\Theta(n)$.

Complexidade assintótica

Definição

A **complexidade computacional** de um algoritmo é a quantidade de recursos computacionais necessária para executá-lo.

Definição

A **complexidade computacional assintótica** de um algoritmo é a estimativa da complexidade computacional utilizando a análise assintótica.

Seja T a função tal que $T(n)$ é o número de operações de um determinado algoritmo dado uma entrada de tamanho n . Analisamos assintoticamente T , ou seja, tentamos calcular $\Theta(T(n))$ ou encontrar funções f tais que $T(n) = O(f(n))$.

- (i) Iterar todos os elementos de um vetor de n elementos tem complexidade $\Theta(n)$.
- (ii) Iterar todos os subconjuntos tem complexidade $\Theta(2^n)$.

Em um algoritmo recursivo, é possível escrever a função de complexidade assintótica como uma função recursiva.

Em um algoritmo recursivo, é possível escrever a função de complexidade assintótica como uma função recursiva.

Temos os seguintes métodos para estimar a complexidade de um algoritmo recursivo:

Em um algoritmo recursivo, é possível escrever a função de complexidade assintótica como uma função recursiva.

Temos os seguintes métodos para estimar a complexidade de um algoritmo recursivo:

- (i) **Indução**: chute a complexidade de T e use o passo indutivo para provar. Pouco útil em Maratonas.

Em um algoritmo recursivo, é possível escrever a função de complexidade assintótica como uma função recursiva.

Temos os seguintes métodos para estimar a complexidade de um algoritmo recursivo:

- (i) **Indução**: chute a complexidade de T e use o passo indutivo para provar. Pouco útil em Maratonas.
- (ii) **Árvore de recursão**: escreva a árvore de recursão de T , com cada nó contendo a complexidade de resolver aquele passo, e faça a soma de todos os nós.

Em um algoritmo recursivo, é possível escrever a função de complexidade assintótica como uma função recursiva.

Temos os seguintes métodos para estimar a complexidade de um algoritmo recursivo:

- (i) **Indução:** chute a complexidade de T e use o passo indutivo para provar. Pouco útil em Maratonas.
- (ii) **Árvore de recursão:** escreva a árvore de recursão de T , com cada nó contendo a complexidade de resolver aquele passo, e faça a soma de todos os nós.
- (iii) **Método Mestre:** se $T(n) = aT(n/b) + f(n)$ para $a \geq 1$ e $b > 1$, o Teorema Mestre nos dá complexidade de T .

Árvore de recursão

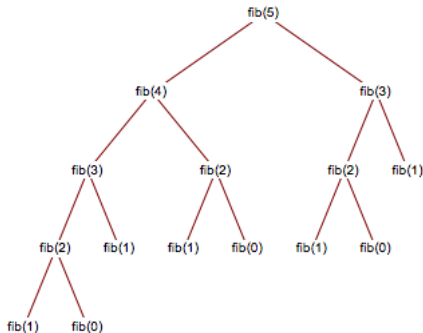
Algoritmo de fibonacci

```
int f(int n) {  
    if(n == 0) return 0;  
    if(n == 1) return 1;  
    return (f(n-1) + f(n-2)) % 998244353; // MOD = 1e9 + 7;  
}
```

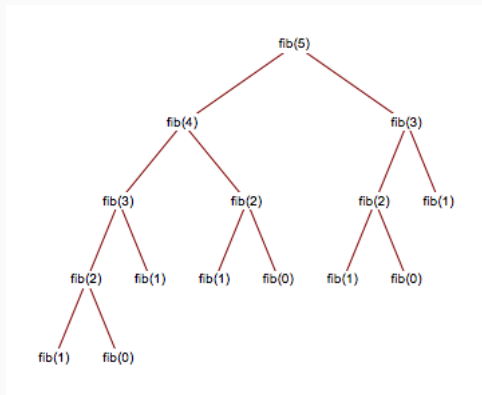
Árvore de recursão

Algoritmo de fibonacci

```
int f(int n) {  
    if(n == 0) return 0;  
    if(n == 1) return 1;  
    return (f(n-1) + f(n-2)) % 998244353; // MOD = 1e9 + 7;  
}
```

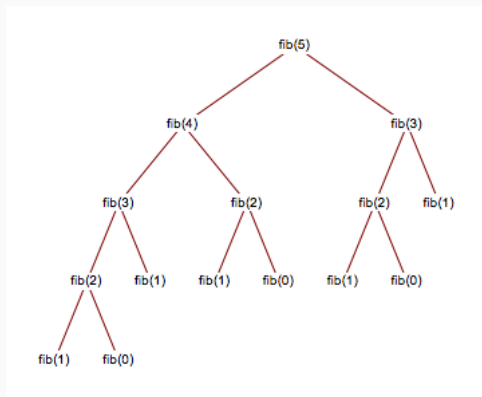


Árvore de recursão



A árvore completa tem 2^n passos, e o custo de cada nó é $O(1)$, então $T(n) = O(2^n)$.

Árvore de recursão



A árvore completa tem 2^n passos, e o custo de cada nó é $O(1)$, então $T(n) = O(2^n)$.

Obs. Uma análise da equação característica nos dá $T(n) = \Theta((1 + \sqrt{5})/2)^n$.

Teorema Mestre

Theorem (Teorema Mestre)

Sejam $a \geq 1$ e $b > 1$ inteiros constante, $f(n)$ uma função não negativa, e $T(n)$ definida nos inteiros não-negativos pela recorrência

$$T(n) = aT(n/b) + f(n) ,$$

interpretando n/b como $\lceil n/b \rceil$ ou $\lfloor n/b \rfloor$. Então $T(n)$ tem os seguintes limites:

Teorema Mestre

Theorem (Teorema Mestre)

Sejam $a \geq 1$ e $b > 1$ inteiros constante, $f(n)$ uma função não negativa, e $T(n)$ definida nos inteiros não-negativos pela recorrência

$$T(n) = aT(n/b) + f(n) ,$$

interpretando n/b como $\lceil n/b \rceil$ ou $\lfloor n/b \rfloor$. Então $T(n)$ tem os seguintes limites:

- (i) *Se $f(n) = O(n^{\log_b a - \varepsilon})$ para alguma constante $\varepsilon > 0$, então $T(n) = \Theta(n^{\log_b a})$.*

Teorema Mestre

Theorem (Teorema Mestre)

Sejam $a \geq 1$ e $b > 1$ inteiros constante, $f(n)$ uma função não negativa, e $T(n)$ definida nos inteiros não-negativos pela recorrência

$$T(n) = aT(n/b) + f(n) ,$$

interpretando n/b como $\lceil n/b \rceil$ ou $\lfloor n/b \rfloor$. Então $T(n)$ tem os seguintes limites:

- (i) Se $f(n) = O(n^{\log_b a - \varepsilon})$ para alguma constante $\varepsilon > 0$, então $T(n) = \Theta(n^{\log_b a})$.
- (ii) Se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \log n)$.

Teorema Mestre

Theorem (Teorema Mestre)

Sejam $a \geq 1$ e $b > 1$ inteiros constante, $f(n)$ uma função não negativa, e $T(n)$ definida nos inteiros não-negativos pela recorrência

$$T(n) = aT(n/b) + f(n) ,$$

interpretando n/b como $\lceil n/b \rceil$ ou $\lfloor n/b \rfloor$. Então $T(n)$ tem os seguintes limites:

- (i) Se $f(n) = O(n^{\log_b a - \varepsilon})$ para alguma constante $\varepsilon > 0$, então $T(n) = \Theta(n^{\log_b a})$.
- (ii) Se $f(n) = \Theta(n^{\log_b a})$, então $T(n) = \Theta(n^{\log_b a} \log n)$.
- (iii) Se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ para alguma constante $\varepsilon > 0$, e se $af(n/b) \leq cf(n)$ para alguma constante $c < 1$ e todo n suficientemente grande, então $T(n) = \Theta(f(n))$.

Exponenciação rápida

Código de exponenciação rápida

```
int fexp(int b, int e) {  
    if(e == 0) return 1;  
    int answer = fexp(b, e/2);  
    answer = (answer * answer);  
    if(e%2 == 1) answer = answer * b;  
    return answer;  
}
```

Exponenciação rápida

Código de exponenciação rápida

```
int fexp(int b, int e) {  
    if(e == 0) return 1;  
    int answer = fexp(b, e/2);  
    answer = (answer * answer);  
    if(e%2 == 1) answer = answer * b;  
    return answer;  
}
```

A complexidade assintótica da função acima é dada pela função $T(n) = T(n/2) + 1$. Temos $\log_2 1 = 0$, logo $f(n) = \Theta(n^0) = \Theta(1)$.

Exponenciação rápida

Código de exponenciação rápida

```
int fexp(int b, int e) {  
    if(e == 0) return 1;  
    int answer = fexp(b, e/2);  
    answer = (answer * answer);  
    if(e%2 == 1) answer = answer * b;  
    return answer;  
}
```

A complexidade assintótica da função acima é dada pela função $T(n) = T(n/2) + 1$. Temos $\log_2 1 = 0$, logo $f(n) = \Theta(n^0) = \Theta(1)$. Pelo Teorema Mestre, vale

$$T(n) = \Theta(\log n) .$$

Exercício

Exponenciação rápida

Código de exponenciação rápida

```
int fexp(int b, int e) {  
    if(e == 0) return 1;  
    int answer = fexp(b, e/2);  
    answer = (answer * answer);  
    if(e%2 == 1) answer = answer * b;  
    return answer;  
}
```

A complexidade assintótica da função acima é dada pela função $T(n) = T(n/2) + 1$. Temos $\log_2 1 = 0$, logo $f(n) = \Theta(n^0) = \Theta(1)$. Pelo Teorema Mestre, vale

$$T(n) = \Theta(\log n) .$$

Exercício

Pesquise o algoritmo de Karatsuba, e calcule sua complexidade assintótica com o Teorema Mestre.

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

O número de maneiras pode ser modelado recursivamente como $f(n) = f(n-1) + f(n-2)$. Vejamos uma abordagem diferente:

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

O número de maneiras pode ser modelado recursivamente como $f(n) = f(n-1) + f(n-2)$. Vejamos uma abordagem diferente:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} f(n) \\ f(n-1) \end{pmatrix} = \begin{pmatrix} f(n) + f(n-1) = f(n+1) \\ f(n) \end{pmatrix}$$

Seja A a matriz acima e $v_n = (f(n), f(n-1))$. A equação acima diz que $Av_n = v_{n+1}$. Como $Av_{n+1} = v_{n+2}$, Temos

$$A(Av_n) = v_{n+2}.$$

Pela associatividade da multiplicação de matrizes, $A^2 v_n = v_{n+2}$. Assim, $A^{10^{18}-1} v_1 = v_{10^{18}}$.

Solução. Use o algoritmo de exponenciação rápida para calcular A^n em $O(\log n)$.

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

O número de maneiras pode ser modelado recursivamente como $f(n) = f(n-1) + f(n-2)$. Vejamos uma abordagem diferente:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} f(n) \\ f(n-1) \end{pmatrix} = \begin{pmatrix} f(n) + f(n-1) = f(n+1) \\ f(n) \end{pmatrix}$$

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

O número de maneiras pode ser modelado recursivamente como $f(n) = f(n-1) + f(n-2)$. Vejamos uma abordagem diferente:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} f(n) \\ f(n-1) \end{pmatrix} = \begin{pmatrix} f(n) + f(n-1) = f(n+1) \\ f(n) \end{pmatrix}$$

Seja A a matriz acima e $v_n = (f(n), f(n-1))$.

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

O número de maneiras pode ser modelado recursivamente como $f(n) = f(n-1) + f(n-2)$. Vejamos uma abordagem diferente:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} f(n) \\ f(n-1) \end{pmatrix} = \begin{pmatrix} f(n) + f(n-1) = f(n+1) \\ f(n) \end{pmatrix}$$

Seja A a matriz acima e $v_n = (f(n), f(n-1))$. A equação acima diz que $Av_n = v_{n+1}$. Como $Av_{n+1} = v_{n+2}$, Temos

$$A(Av_n) = v_{n+2}.$$

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

O número de maneiras pode ser modelado recursivamente como $f(n) = f(n-1) + f(n-2)$. Vejamos uma abordagem diferente:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} f(n) \\ f(n-1) \end{pmatrix} = \begin{pmatrix} f(n) + f(n-1) = f(n+1) \\ f(n) \end{pmatrix}$$

Seja A a matriz acima e $v_n = (f(n), f(n-1))$. A equação acima diz que $Av_n = v_{n+1}$. Como $Av_{n+1} = v_{n+2}$, Temos

$$A(Av_n) = v_{n+2}.$$

Pela associatividade da multiplicação de matrizes, $A^2 v_n = v_{n+2}$. Assim, $A^{10^{18}-1} v_1 = v_{10^{18}}$.

Um problema não trivial

Problema

De quantas maneiras podemos preencher um tabuleiro $1 \times n$ com peças 1×1 e 1×2 ?

O número de maneiras pode ser modelado recursivamente como $f(n) = f(n-1) + f(n-2)$. Vejamos uma abordagem diferente:

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} f(n) \\ f(n-1) \end{pmatrix} = \begin{pmatrix} f(n) + f(n-1) = f(n+1) \\ f(n) \end{pmatrix}$$

Seja A a matriz acima e $v_n = (f(n), f(n-1))$. A equação acima diz que $Av_n = v_{n+1}$. Como $Av_{n+1} = v_{n+2}$, Temos

$$A(Av_n) = v_{n+2}.$$

Pela associatividade da multiplicação de matrizes, $A^2v_n = v_{n+2}$. Assim, $A^{10^{18}-1}v_1 = v_{10^{18}}$.

Solução. Use o algoritmo de exponenciação rápida para calcular A^n em $O(\log n)$.

Uma complexidade notável

Problema

Encontra quantos primos existem de 1 até n , para $n \leq 10^6$.

Uma complexidade notável

Problema

Encontra quantos primos existem de 1 até n , para $n \leq 10^6$.

É possível testar se um número é primo em $O(\sqrt{n})$, mas testar todos os números de 1 até n terá aproximadamente 10 segundos de execução.

Uma complexidade notável

Problema

Encontra quantos primos existem de 1 até n , para $n \leq 10^6$.

É possível testar se um número é primo em $O(\sqrt{n})$, mas testar todos os números de 1 até n terá aproximadamente 10 segundos de execução.

Uma outra abordagem é, para cada número, marcar que seus múltiplos não são primos.

Uma complexidade notável

Problema

Encontra quantos primos existem de 1 até n , para $n \leq 10^6$.

É possível testar se um número é primo em $O(\sqrt{n})$, mas testar todos os números de 1 até n terá aproximadamente 10 segundos de execução.

Uma outra abordagem é, para cada número, marcar que seus múltiplos não são primos.

Crivo de eratóstenes

```
for(int i = 2; i <= n; i++) {  
    for(int j = i+i; j <= n; j+=i) {  
        primo[j] = 0;  
    }  
}
```

A complexidade do algoritmo acima é:

Uma complexidade notável

Problema

Encontra quantos primos existem de 1 até n , para $n \leq 10^6$.

É possível testar se um número é primo em $O(\sqrt{n})$, mas testar todos os números de 1 até n terá aproximadamente 10 segundos de execução.

Uma outra abordagem é, para cada número, marcar que seus múltiplos não são primos.

Crivo de eratóstenes

```
for(int i = 2; i <= n; i++) {  
    for(int j = i+i; j <= n; j+=i) {  
        primo[j] = 0;  
    }  
}
```

A complexidade do algoritmo acima é: $O(n \log n)$.